JAVA COLLECTIONS FRAMEWORK

Complete Interview Question Bank | Beginner to Advanced | 3-5 Years Experience

How to use this guide: Prepare fundamentals first, then HashMap internals and concurrent collections. Practise the coding and scenario-based questions in an IDE.

1. Collection Framework Fundamentals

1. What is the Java Collection Framework?
2. Why do we use Collections in Java?
3. What are the advantages of the Collection Framework?
4. Explain the hierarchy of the Collection Framework.
5. What is the difference between Collection and Collections?
6. What is the difference between Collection and Map?
7. What are the main interfaces in the Collection Framework?
8. What is the difference between List, Set, Queue and Map?
9. Is Collection an interface or a class?
10. Why is Map not a child interface of Collection?
11. What is the difference between an Array and an ArrayList?
12. What is the difference between Iterator and ListIterator?
13. What is the difference between Iterator and Enumeration?
14. What is the difference between fail-fast and weakly consistent iterators?
15. What is the difference between Comparable and Comparator?

2. List Interface

16. What is List in Java?
17. What are the implementations of List?
18. What is ArrayList?
19. How does ArrayList work internally?
20. What is the default initial capacity of ArrayList?
21. How does ArrayList increase its capacity?
22. What is the time complexity of ArrayList operations?
23. Why is ArrayList efficient for random access?
24. What is LinkedList?
25. How does LinkedList work internally?
26. What is the difference between ArrayList and LinkedList?
27. When should we use ArrayList and LinkedList?
28. Is ArrayList thread-safe?
29. How can we make an ArrayList synchronized?
30. What is Vector?
31. What is the difference between ArrayList and Vector?
32. What is Stack in Java?

33. Why is Stack considered a legacy class?
34. What is the difference between Stack and ArrayDeque?
35. Can ArrayList store duplicate and null values?

3. Set Interface

36. What is Set in Java?
37. Why does Set not allow duplicate elements?
38. What are the implementations of Set?
39. What is HashSet?
40. How does HashSet work internally?
41. How does HashSet identify duplicate elements?
42. What is the role of hashCode() and equals() in HashSet?
43. Can HashSet store null values?
44. What is LinkedHashSet?
45. What is the difference between HashSet and LinkedHashSet?
46. What is TreeSet?
47. How does TreeSet work internally?
48. What is the difference between HashSet and TreeSet?
49. What is the difference between LinkedHashSet and TreeSet?
50. Does TreeSet allow null values?
51. How does TreeSet maintain sorted order?
52. What happens if we insert duplicate elements into a Set?
53. Can we store custom objects in HashSet?
54. What happens if we override equals() but not hashCode()?
55. What happens if we modify an object after inserting it into HashSet?

4. Map Interface

56. What is Map in Java?
57. What are the implementations of Map?
58. What is HashMap?
59. How does HashMap work internally?
60. How does HashMap store key-value pairs?
61. What is hashing?
62. What is a hash collision?
63. How does HashMap handle collisions?
64. What is the role of hashCode() and equals() in HashMap?
65. Can HashMap store null keys and null values?
66. Can HashMap contain duplicate keys?
67. What happens when we insert an existing key?
68. What is the default initial capacity of HashMap?
69. What is the default load factor of HashMap?
70. What happens when the HashMap threshold is reached?

71. What is the time complexity of HashMap operations?
72. What is the difference between HashMap and Hashtable?
73. What is the difference between HashMap and ConcurrentHashMap?
74. What is LinkedHashMap?
75. What is the difference between HashMap and LinkedHashMap?
76. What is TreeMap?
77. How does TreeMap work internally?
78. What is the difference between HashMap and TreeMap?
79. Does TreeMap allow null keys?
80. How does TreeMap maintain sorted order?
81. What is WeakHashMap?
82. What is IdentityHashMap?
83. What is EnumMap?
84. What is the difference between put() and putIfAbsent()?
85. What is the difference between get() and getOrDefault()?

5. HashMap Internal Working - High Priority

86. Explain the complete internal working of HashMap.
87. How is the hash calculated in HashMap?
88. How is the bucket index calculated?
89. Why does HashMap use a power-of-two capacity?
90. What is the default capacity of HashMap?
91. What is the load factor?
92. What is the threshold?
93. What is rehashing?
94. What is resizing?
95. How does HashMap handle hash collisions?
96. What is a linked list in a HashMap bucket?
97. When does a linked list convert into a Red-Black Tree?
98. What is treeification?
99. What is untreeification?
100. Why was the Red-Black Tree introduced in Java 8?
101. What is the time complexity before and after treeification?
102. What happens if two different keys have the same hashCode?
103. What happens if two objects have the same hashCode and equals() returns false?
104. What happens if equals() returns true but hashCode() is different?
105. Why should immutable objects be preferred as HashMap keys?

6. Queue and Deque

106. What is Queue in Java?
107. What is FIFO?
108. What are the implementations of Queue?

109. What is PriorityQueue?
110. How does PriorityQueue work internally?
111. What is a Min Heap?
112. What is a Max Heap?
113. How can we implement a Max Heap using PriorityQueue?
114. What is the difference between PriorityQueue and LinkedList?
115. What is BlockingQueue?
116. What is ArrayDeque?
117. What is Deque?
118. What is the difference between Queue and Deque?
119. What is the difference between poll() and remove()?
120. What is the difference between peek() and element()?
121. What is the difference between offer() and add()?
122. Can PriorityQueue contain null?
123. Does PriorityQueue maintain insertion order?

7. Comparable and Comparator

124. What is Comparable?
125. What is Comparator?
126. What is the difference between Comparable and Comparator?
127. Why do we use compareTo()?
128. Why do we use compare()?
129. How do we sort custom objects?
130. How can we sort employees by salary?
131. How can we sort employees by name?
132. How can we sort by multiple fields?
133. What is natural ordering?
134. What is custom ordering?
135. What is the difference between Collections.sort() and List.sort()?
136. How do we sort in descending order?
137. What is Comparator.comparing()?
138. What is thenComparing()?

8. Iterator and Concurrent Modification

139. What is Iterator?
140. How does Iterator work internally?
141. What is ListIterator?
142. What is ConcurrentModificationException?
143. When does ConcurrentModificationException occur?
144. How can we safely remove elements while iterating?
145. What is the difference between fail-fast and weakly consistent iterators?
146. Is Iterator thread-safe?

147. What is the modCount variable?
148. Why does ConcurrentHashMap iterator generally not throw ConcurrentModificationException during concurrent updates?

9. Multithreading and Concurrent Collections

149. What is a thread-safe collection?
150. Why is HashMap not thread-safe?
151. What is ConcurrentHashMap?
152. How does ConcurrentHashMap work internally?
153. How did ConcurrentHashMap change between Java 7 and Java 8?
154. Why is ConcurrentHashMap better than Hashtable?
155. Can ConcurrentHashMap store null keys or values?
156. What is CopyOnWriteArrayList?
157. How does CopyOnWriteArrayList work internally?
158. When should we use CopyOnWriteArrayList?
159. What is CopyOnWriteArraySet?
160. What is BlockingQueue?
161. What is ArrayBlockingQueue?
162. What is LinkedBlockingQueue?
163. What is the difference between ArrayBlockingQueue and LinkedBlockingQueue?
164. What is SynchronousQueue?
165. What is ConcurrentLinkedQueue?
166. What is the difference between synchronizedList() and CopyOnWriteArrayList?
167. What is Collections.synchronizedMap()?
168. What is the difference between ConcurrentHashMap and Collections.synchronizedMap()?

10. Java 8 Collection Features

169. What is forEach() in Collection?
170. What is the removeIf() method?
171. What is the replaceAll() method?
172. What is the Stream API?
173. What is the difference between map() and flatMap()?
174. What is filter()?
175. What is collect()?
176. What is Collectors.toList()?
177. What is Collectors.toMap()?
178. What happens when duplicate keys occur in Collectors.toMap()?
179. How do we convert a List into a Map?
180. How do we convert a List into a Set?
181. How do we sort a List using Java 8?
182. How do we remove duplicates using Stream API?
183. What is the difference between stream() and parallelStream()?

11. Java 9, 10 and 21 Collection Features

184. How do you create an unmodifiable List using List.of()?
185. What is the difference between Arrays.asList() and List.of()?
186. What is the difference between an unmodifiable collection and an immutable collection?
187. What are Set.of() and Map.of()?
188. What is copyOf() in collection interfaces?
189. What is var in Java 10?
190. What are Sequenced Collections in Java 21?
191. What is the SequencedCollection interface?
192. What is SequencedSet?
193. What is SequencedMap?
194. What are getFirst(), getLast() and reversed()?
195. How do Sequenced Collections improve collection handling?

12. Collections Utility Class

196. What is the Collections utility class?
197. What is Collections.sort()?
198. What is Collections.reverse()?
199. What is Collections.shuffle()?
200. What is Collections.frequency()?
201. What are Collections.max() and min()?
202. What is Collections.binarySearch()?
203. What is Collections.unmodifiableList()?
204. What is Collections.synchronizedList()?
205. What is Collections.singletonList()?
206. What is Collections.emptyList()?
207. What is the difference between emptyList() and new ArrayList()?

13. Time Complexity

208. What is the time complexity of ArrayList search, insertion and deletion?
209. What is the time complexity of LinkedList operations?
210. What is the average time complexity of HashSet operations?
211. What is the time complexity of TreeSet operations?
212. What is the average time complexity of HashMap operations?
213. What is the time complexity of TreeMap operations?
214. What is the time complexity of PriorityQueue operations?
215. What is amortized time complexity?
216. Why is ArrayList insertion at the beginning O(n)?
217. Why is HashMap average lookup O(1)?
218. Why is TreeMap lookup O(log n)?
219. What is the worst-case complexity of HashMap lookup?

14. Practical Coding Interview Questions

220. Remove duplicate elements from a List.
221. Find duplicate elements in an ArrayList.
222. Find the first non-repeating character in a String.
223. Find the first repeating character in a String.
224. Count the frequency of each character using HashMap.
225. Count the frequency of each word using HashMap.
226. Sort a HashMap by keys.
227. Sort a HashMap by values.
228. Find the second-highest salary from an Employee List.
229. Find the highest-paid employee in each department.
230. Convert a List of Employees into a Map using Stream API.
231. Group employees by department.
232. Find duplicate employees based on employee ID.
233. Merge two Lists and remove duplicates.
234. Find common elements between two Lists.
235. Find missing numbers using Set.
236. Find the intersection and union of two Sets.
237. Sort employees by salary and name.
238. Find the top 3 highest salaries.
239. Implement an LRU Cache using LinkedHashMap.
240. Implement a frequency counter using HashMap.
241. Reverse an ArrayList without using Collections.reverse().
242. Find the kth largest element using PriorityQueue.
243. Find the most frequent element in an array.
244. Design a thread-safe cache using ConcurrentHashMap.

15. Scenario-Based Questions - 3 to 5 Years Experience

245. You need to store unique employee IDs. Which collection will you use and why?
246. You need to maintain insertion order and avoid duplicates. Which collection will you use?
247. You need to store employee records sorted by employee ID. Which collection will you use?
248. You need fast key-value lookup. Which collection will you use?
249. You need a thread-safe key-value store. Which collection will you use?
250. You need to implement an LRU cache. Which collection will you use?
251. You have 10 lakh records. Which collection would you choose for searching, and why?
252. You need to process tasks based on priority. Which collection will you use?
253. You need to maintain FIFO order in a producer-consumer system. Which collection will you use?
254. You need frequent read operations and very few write operations. Which collection will you consider?
255. Multiple threads are updating a shared Map. What will you use?
256. You need to maintain sorted unique values. Which collection will you use?
257. You need to store custom objects as HashMap keys. What precautions will you take?

258. You need to remove elements during iteration. How will you do it safely?

259. You need to cache API responses in a Spring Boot application. Which data structure would you consider?

Important Java 8+ HashMap reference values

• Default initial capacity: 16

• Default load factor: 0.75

• Default resize threshold at capacity 16: 12

• Treeification threshold: 8 entries in a bin, subject to table capacity

• Minimum table capacity for treeification: 64

• Untreeification threshold is implementation-dependent in the tree-bin removal path; commonly referenced value: 6

Note: Internal implementation details can vary by Java version. Verify version-specific behavior when answering deep implementation questions.